
Path Recreater

Mathematics Breakdown

2025 Azerbaijan Grand Prix | DUSt3R / MonST3R Framework

1. Notation and Definitions

This section defines every mathematical symbol used throughout this document. All notation is consistent with the multi-view geometry and SLAM literature.

- **I** — An image frame. I in $\mathbb{R}^{H \times W \times 3}$ where H = height in pixels, W = width in pixels, 3 = RGB colour channels.
- **I_a, I_b** — Two distinct frames forming a processing pair.
- **X** — A pointmap. X in $\mathbb{R}^{H \times W \times 3}$. For each pixel (u,v) , $X(u,v)$ in $\mathbb{R}^3$ is the predicted 3D coordinate of the scene point visible at that pixel.
- **C** — A confidence map. C in $\mathbb{R}^{H \times W}$. $C(u,v)$ in $\mathbb{R}^+$ is the network's certainty about the 3D prediction at pixel (u,v) . Higher = more certain.
- **T** — A rigid body transform (pose). T in $SE(3)$, the Special Euclidean group in 3D. Represented as a 4×4 matrix.
- **R** — Rotation matrix. R in $SO(3)$: $\det(R) = 1$, $R^T R = I_3$.
- **t** — Translation vector. t in $\mathbb{R}^3$.
- **s** — Scalar scale factor. s in $\mathbb{R}^+$. Required because monocular reconstruction has no absolute depth reference.
- **sigma** — Per-pair scale variable used in global alignment. σ_{ij} in $\mathbb{R}^+$.
- **chi** — Canonical pointmap (the fused 3D scene). χ in $\mathbb{R}^{H \times W \times 3}$.
- **E** — Global alignment energy. E in $\mathbb{R}^+$. The objective function to be minimised.
- **M** — Binary confidence mask. $M(u,v)$ in $\{0,1\}$. Derived by thresholding C at a percentile threshold τ .
- **tau** — Confidence percentile threshold. τ in $[0,1]$. Controls how many pixels are kept.
- **C_{bar}_{tau}** — The τ -th percentile of confidence values in a chunk. Used as the binary threshold.
- **n** — Number of frames in a chunk.
- **c** — Chunk size (number of frames per chunk). $c = 20$.
- **o** — Overlap between consecutive chunks. $o = 5$.
- **s_t** — Stride between chunk starts. $s_t = c - o = 15$.
- **f_{theta}** — The DUSt3R / MonST3R network with fixed pretrained weights θ .
- **p** — A 3D point in homogeneous coordinates: $p = (x, y, z, 1)^T$ in $\mathbb{R}^4$.
- **mu_x, mu_y** — Mean of source and target point sets respectively.
- **Sigma_{xy}** — Cross-covariance matrix between source and target point sets.
- **U, D, V^T** — SVD decomposition of Σ_{xy} .

-
- **S** — Sign-correction diagonal matrix used in Umeyama: $S = \text{diag}(1, 1, \det(U) \cdot \det(V))$.

2. Video Preprocessing — Mathematical Foundations

Before any reconstruction can occur, the raw video must be transformed into a format that the network can process. Each transformation has a precise mathematical definition and a geometric justification.

2.1 Spatial Downscaling — Bilinear Interpolation

The raw frame I in $\mathbb{R}^{1080 \times 1920 \times 3}$ is too large for the GPU memory budget. We rescale it to a target resolution using bilinear interpolation. Let $r_H = H_{out} / H_{in}$ and $r_W = W_{out} / W_{in}$ be the vertical and horizontal scale factors. The output pixel at position (x', y') is computed as:

$$\begin{aligned} x_f &= x' / r_W, & y_f &= y' / r_H \\ x_0 &= \text{floor}(x_f), & y_0 &= \text{floor}(y_f), & x_1 &= x_0 + 1, & y_1 &= y_0 + 1 \\ w_x &= x_f - x_0, & w_y &= y_f - y_0 \\ I'(x', y') &= (1-w_x)(1-w_y) I(x_0, y_0) + w_x(1-w_y) I(x_1, y_0) \\ &\quad + (1-w_x) w_y I(x_0, y_1) + w_x w_y I(x_1, y_1) \end{aligned}$$

The four weights $(1-w_x)(1-w_y)$, $w_x(1-w_y)$, $(1-w_x)w_y$, $w_x w_y$ are the bilinear interpolation coefficients. They sum to 1 by construction, preserving the mean intensity of the image. The scaling we applied:

$$\begin{aligned} (H_{in}, W_{in}) &= (1080, 1920) \quad \rightarrow \quad (H_{out}, W_{out}) = (576, 1024) \\ r_H &= 576 / 1080 = 0.533, & r_W &= 1024 / 1920 = 0.533 \end{aligned}$$

Note: The DUST3R network further downscales internally to 512 px on the longest side. Our choice of 1024 wide is therefore the effective maximum useful resolution before the network discards the extra detail.

2.2 Frame Rate Reduction — Temporal Subsampling

We keep every k -th frame from the 50 fps video, reducing the effective frame rate to approximately 24 fps. Let $F = \{f_0, f_1, \dots, f_{N-1}\}$ be the full frame sequence. The subsampled sequence is:

$$\begin{aligned} F' &= \{f_{k \cdot i} : i = 0, 1, 2, \dots\} \quad \text{where } k = 2 \\ \text{fps}_{out} &= \text{fps}_{in} / k = 50 / 2 = 25 \text{ fps} \quad (\text{approximately 24 fps target}) \end{aligned}$$

The geometric motivation is the parallax baseline — the distance the camera travels between two consecutive frames. This determines whether depth can be reliably estimated:

$$\text{baseline}(\text{fps}) = v * \text{delta}_t = v / \text{fps}$$

where v is the camera speed in m/s. At $v = 200 \text{ km/h} = 55.6 \text{ m/s}$:

$$\begin{aligned} \text{baseline}(50 \text{ fps}) &= 55.6 / 50 = 1.11 \text{ m} \quad (\text{too small for reliable depth}) \\ \text{baseline}(24 \text{ fps}) &= 55.6 / 24 = 2.32 \text{ m} \quad (\text{optimal range}) \\ \text{baseline}(10 \text{ fps}) &= 55.6 / 10 = 5.56 \text{ m} \quad (\text{too large, matching fails}) \end{aligned}$$

The optical flow estimator (RAFT, used internally by MonST3R) is trained to handle displacements up to approximately 3-4 metres between frames at this scale. Below 12 fps the baseline exceeds this range and correspondence estimation breaks down catastrophically.

$$\text{fps}_{min} = v / \text{baseline}_{max} = 55.6 / 4.0 = 13.9 \text{ fps}$$

We use 12 fps as the practical floor, corresponding to a maximum baseline of approximately 4.6 m.

2.3 Frame Cropping — Removing the Static Foreground

The car body occupies the lower fraction of every frame. Since the car is rigidly attached to the camera, it never moves relative to the image plane. Define the crop operation on a frame I in $R^{\{H \times W \times 3\}}$ as:

$$I_{crop}(x, y) = I(x, y) \quad \text{for} \quad 0 \leq y < H/2$$

$$I_{crop} \text{ in } R^{\{(H/2) \times W \times 3\}}$$

For our 576-pixel tall frames after downscaling: $H/2 = 288$. The cropped frame is 1024×288 pixels containing only the scene ahead — the road, barriers, buildings, and sky.

The mathematical reason cropping outperforms masking (setting lower pixels to zero) lies in the confidence-weighted energy function used in global alignment. If we mask, then for every masked pixel (u,v) :

$$I_{mask}(x, y) = 0 \quad \text{for} \quad y \geq H/2$$

The network predicts low confidence at constant-zero regions: $C(u,v) \rightarrow 1$ (minimum) for masked pixels. In the alignment energy:

$$E = \text{Sum}_{\{pixels\}} C(u,v) * ||T^*X(u,v) - chi(u,v)||^2$$

Masked pixels contribute terms with $C(u,v)$ approx 1 (minimum weight) but still consume computational resources and pollute the point cloud with low-confidence junk points near zero. Cropping removes them from the input entirely, so the model allocates 100% of its capacity to valid scene pixels. The effective information density per compute unit doubles.

2.4 Overlapping Chunking — Window Construction

The full frame sequence $F' = \{f_0, f_1, \dots, f_{\{N-1\}}\}$ ($N = 880$ frames after subsampling) must be divided into manageable windows. The j -th chunk is defined as:

$$C_j = \{ f_{\{j*s_t\}}, f_{\{j*s_t + 1\}}, \dots, f_{\{j*s_t + c - 1\}} \}$$

$$\text{where } s_t = c - o = 20 - 5 = 15 \quad (\text{stride})$$

$$\text{Total chunks} = \text{ceil}((N - c) / s_t) + 1 = \text{ceil}((880 - 20) / 15) + 1$$

$$= 58 + 1 = 59 \text{ chunks}$$

The overlap of $o = 5$ frames between consecutive chunks is the stitching mechanism. Chunk j and chunk $j+1$ share exactly o frames:

$$C_j \text{ INTERSECT } C_{\{j+1\}} = \{ f_{\{(j+1)*s_t\}}, \dots, f_{\{(j+1)*s_t + o - 1\}} \}$$

$$|C_j \text{ INTERSECT } C_{\{j+1\}}| = o = 5 \text{ frames}$$

The minimum overlap required for stitching via the Umeyama method is 3 frames (the minimum for a well-determined similarity transform in R^3). We use 5 for robustness.

3. Pairwise Pointmap Prediction

This is the core inference step. The DUS3R network takes two frames and predicts the full 3D structure of the scene visible in both, without any explicit feature detection or matching.

3.1 The Prediction Formula

For any two frames I_a and I_b from the same chunk, the network f_{θ} predicts:

$$(X_a, X_b, C_a, C_b) = f_{\theta}(I_a, I_b)$$

where:

- X_a in $\mathbb{R}^{H \times W \times 3}$ — the 3D coordinate of the scene point at each pixel of I_a , expressed in I_a 's camera frame
- X_b in $\mathbb{R}^{H \times W \times 3}$ — the 3D coordinate of the scene point at each pixel of I_b , ALSO expressed in I_a 's camera frame
- C_a in $\mathbb{R}^{H \times W}$ — per-pixel confidence for X_a predictions
- C_b in $\mathbb{R}^{H \times W}$ — per-pixel confidence for X_b predictions

The key property: both X_a and X_b are in I_a 's coordinate system. This means the relative geometry between the two cameras is implicitly encoded in the two pointmaps without needing to estimate a fundamental matrix or run RANSAC.

3.2 Network Output Activation Functions

The depth values in the pointmap use an exponential activation to enforce positivity:

$$X(u, v) = \exp(F_{\text{decoder}}(u, v))$$

where $F_{\text{decoder}}(u, v)$ in $\mathbb{R}^3$ is the raw decoder output at pixel (u, v) . Since $\exp(x) > 0$ for all real x , this guarantees $X(u, v)_z > 0$ (points are in front of the camera).

The confidence activation adds a minimum floor of 1:

$$C(u, v) = \exp(G_{\text{decoder}}(u, v)) + 1$$

This ensures $C(u, v)$ in $(1, \infty)$ always. The minimum confidence of 1 is the neutral weight; high confidence points have $C \gg 1$ and dominate the alignment energy.

3.3 Pair Generation

For a chunk with n frames, we generate all symmetric pairs. The total number of pairs is:

$$\text{num_pairs} = n * (n-1) / 2 * 2 = n * (n-1)$$

(The factor of 2 accounts for symmetry: pair (a, b) and pair (b, a) are both generated, since the network is not symmetric in its two inputs.) For $n = 12$ frames:

$$\text{num_pairs} = 12 * 11 = 132 \text{ pairs}$$

The GPU memory required scales as $O(\text{num_pairs} * H * W)$ for storing all pointmaps simultaneously during global alignment. This is why n is bounded by GPU memory — on a 16 GB GPU the practical limit is approximately $n = 12-15$ frames.

3.4 MonST3R Extension — Optical Flow Conditioning

MonST3R extends DUST3R by conditioning on a dense optical flow field $F_{\{a \rightarrow b\}}$ in $\mathbb{R}^{H \times W \times 2}$, estimated by the RAFT network:

$$(X_a, X_b, C_a, C_b, M_a, M_b) = f_{\{MonST3R\}}(I_a, I_b, F_{\{a \rightarrow b\}})$$

The additional outputs M_a, M_b in $[0,1]$ are motion probability masks:

$$\begin{aligned} M_a(u, v) &\approx 1 && \text{if pixel } (u, v) \text{ in } I_a \text{ belongs to a moving object} \\ M_a(u, v) &\approx 0 && \text{if pixel } (u, v) \text{ in } I_a \text{ belongs to the static scene} \end{aligned}$$

The optical flow field is computed as:

$$F_{\{a \rightarrow b\}}(u, v) = (u'(u, v) - u, v'(u, v) - v)$$

where $(u'(u, v), v'(u, v))$ is the position in I_b of the scene point seen at (u, v) in I_a . Pixels where the flow cannot be explained by the camera's rigid motion are flagged as $M \approx 1$ (dynamic objects — other cars).

4. Global Alignment — Energy Minimisation

Global alignment fuses the set of pairwise pointmaps into one coherent, consistent 3D reconstruction. This is the most mathematically involved step.

4.1 The Alignment Energy

Given the set of all pairwise predictions $\{X^{ij}, C^{ij}\}$ for a chunk of n frames, we seek camera poses $\{T_i\}$, per-pair scales $\{\sigma_{ij}\}$, and canonical points χ that minimise the confidence-weighted reprojection energy:

$$E(T, \sigma, \chi) = \sum_{\{(i,j) \text{ in pairs}\}} \sum_{\{(u,v) \text{ in pixels}\}} C^{ij}(u,v) * || T_i * (\sigma_{ij} * X^{ij}(u,v)) - \chi(u,v) ||^2$$

Each term in the sum penalises the discrepancy between: (a) the pointmap prediction $X^{ij}(u,v)$ for frame pair (i,j) , scaled by σ_{ij} and transformed into the canonical frame by T_i , and (b) the canonical point $\chi(u,v)$. The confidence $C^{ij}(u,v)$ weights each pixel's contribution — high-confidence predictions dominate.

4.2 The Rigid Transform Action

The rigid transform T_i in $SE(3)$ is a 4x4 matrix acting on homogeneous coordinates:

$$T_i = \begin{bmatrix} R_i & t_i \\ 0 & 1 \end{bmatrix} \quad \text{where } R_i \text{ in } SO(3), \quad t_i \text{ in } R^3$$
$$T_i * p = R_i * p_{xyz} + t_i \quad \text{for a 3D point } p = (x, y, z)^T$$

The constraint that R_i must be a valid rotation matrix is:

$$R_i^T R_i = I_3 \quad (\text{orthogonality})$$
$$\det(R_i) = +1 \quad (\text{proper rotation, no reflection})$$

During optimisation, the rotation is parameterised to automatically satisfy these constraints (typically via exponential map or quaternion parameterisation).

4.3 The Scale Variable

The scale $\sigma_{ij} > 0$ for each pair accounts for the monocular depth ambiguity. Monocular networks predict relative depth, not absolute depth. Two different pairs may predict the same physical structure at different scales. The σ variables allow the optimiser to normalise all pairs to a common scale.

To enforce positivity, σ is parameterised as:

$$\sigma_{ij} = \exp(s_{ij}) \quad \text{where } s_{ij} \text{ in } R \text{ is unconstrained}$$

This guarantees $\sigma_{ij} > 0$ for any real value of s_{ij} .

4.4 Optimisation Algorithm

The energy E is minimised iteratively using the Adam optimiser (Adaptive Moment Estimation). Adam updates each parameter θ using:

$$m_k = \beta_1 * m_{k-1} + (1 - \beta_1) * \text{grad}_k \quad (\text{first moment})$$
$$v_k = \beta_2 * v_{k-1} + (1 - \beta_2) * \text{grad}_k^2 \quad (\text{second moment})$$

$$\begin{aligned}
m_hat_k &= m_k / (1 - beta_1^k) && (bias \\
&&& correction) \\
v_hat_k &= v_k / (1 - beta_2^k) && (bias \\
&&& correction) \\
theta_k &= theta_{\{k-1\}} - lr_k * m_hat_k / (sqrt(v_hat_k) + epsilon)
\end{aligned}$$

Standard hyperparameters: beta_1 = 0.9, beta_2 = 0.999, epsilon = 1e-8.

The learning rate follows a cosine decay schedule over K = 300 iterations:

$$\begin{aligned}
lr(k) &= lr_0 * (1 + cos(pi * k / K)) / 2 \\
lr_0 &= 0.01, \quad K = 300 \\
lr(0) &= 0.01, \quad lr(150) = 0.005, \quad lr(300) \text{ approx } 0
\end{aligned}$$

4.5 MST Initialisation

Adam requires a good starting point. The Minimum Spanning Tree (MST) initialisation builds an initial solution before Adam runs. It constructs a graph $G = (V, E)$ where:

- **Vertices V** — the n frames in the chunk
- **Edges E** — all processed pairs, each with weight = mean confidence of the pair

The MST of G (found via Prim's or Kruskal's algorithm, $O(|E| \log |V|)$) selects the subset of pairs with highest confidence. These pairs form a spanning tree through all frames. The tree is traversed to propagate poses frame by frame, giving an initial solution that is already approximately correct before Adam begins.

4.6 Binary Confidence Mask

After alignment, the continuous confidence values are thresholded to binary:

$$\begin{aligned}
M(u, v) &= 1 \quad \text{if } C(u, v) \geq \text{Quantile_tau}(C_chunk) \\
M(u, v) &= 0 \quad \text{otherwise}
\end{aligned}$$

The retained points are:

$$P_keep = \{ chi(u, v) : M(u, v) = 1 \}$$

For tau = 0 (our final setting), all finite, valid points are retained. The binary mask then serves as the input selector for the final point cloud: M = 1 means include, M = 0 means exclude. This is the 0-and-1 encoding of the data.

5. Chunk Stitching — Rigid Transform and Similarity Transform

After each chunk is independently aligned, the chunks must be placed in one common world coordinate frame. This section derives both stitching methods used.

5.1 Rigid Weld Formula

Chunk j has camera poses $\{T_{i^{(j)}}\}$ expressed in chunk j 's own coordinate frame. The last overlap frame of chunk j , say frame index i^* , has world-frame pose $T_{i^*}^{\{global\}}$ (established when chunk j was placed in the global frame). The first frame of chunk $j+1$ is the same physical frame i^* , with local pose $T_{i^*}^{\{local\}}$ in chunk $j+1$'s frame. The transform that maps chunk $j+1$ into the global frame is:

$$T_{align} = T_{i^*}^{\{global\}} * (T_{i^*}^{\{local\}})^{-1}$$

The inverse of a SE(3) matrix is:

$$T^{-1} = \begin{bmatrix} R^T & -R^T t \\ 0 & 1 \end{bmatrix}$$

Every pose of chunk $j+1$ becomes:

$$T_{i^{\{world\}}} = T_{align} * T_{i^{\{local\}}}$$

Every 3D point p in chunk $j+1$'s cloud becomes:

$$p_{world} = R_{align} * p_{local} + t_{align}$$

where $T_{align} = (R_{align}, t_{align})$. This is chained across all chunks:

$$T_{align}^{\{(j+1)\}} = T_{i^*}^{\{global, (j)\}} * (T_{i^*}^{\{local, (j+1)\}})^{-1}$$

5.2 Umeyama Similarity Transform — Full Derivation

When per-chunk scale varies, we solve for a similarity transform (s, R, t) that minimises the mean squared distance between n corresponding camera centres. Let x_i (source, local frame) and y_i (target, global frame) be corresponding camera centres for $i = 1, \dots, n$.

Step 1: Compute means.

$$\begin{aligned} \mu_x &= (1/n) \sum_{i=1}^n x_i \\ \mu_y &= (1/n) \sum_{i=1}^n y_i \end{aligned}$$

Step 2: Centre the point sets.

$$\begin{aligned} x_{i_c} &= x_i - \mu_x \\ y_{i_c} &= y_i - \mu_y \end{aligned}$$

Step 3: Compute cross-covariance matrix.

$$\Sigma_{xy} = (1/n) \sum_{i=1}^n y_{i_c} * x_{i_c}^T \quad \text{in } R^{\{3 \times 3\}}$$

Step 4: SVD decomposition.

$$U \ D \ V^T = \text{SVD}(\Sigma_{xy})$$

where $U \in \mathbb{R}^{3 \times 3}$, $D = \text{diag}(d_1, d_2, d_3)$ with $d_1 \geq d_2 \geq d_3 \geq 0$, $V \in \mathbb{R}^{3 \times 3}$.

Step 5: Sign correction to prevent reflection.

$$S = \text{diag}(1, 1, \det(U) * \det(V))$$

If $\det(U) * \det(V) = -1$, S flips the sign of the smallest singular vector to ensure the resulting R has $\det(R) = +1$ (proper rotation). Without this, the solver may return a reflection ($\det(R) = -1$) which looks correct numerically but is geometrically invalid.

Step 6: Compute optimal rotation.

$$R^* = U * S * V^T$$

Verification: $\det(R^*) = \det(U) * \det(S) * \det(V^T) = \det(U) * \det(V) * \det(S)$. Since $\det(S) = \det(U) * \det(V)$, we get $\det(R^*) = (\det(U) * \det(V))^2 = 1$ always.

Step 7: Compute source variance.

$$\sigma_x^2 = (1/n) \sum_{i=1}^n ||x_{i_c}||^2$$

Step 8: Compute optimal scale.

$$s^* = \text{tr}(D * S) / \sigma_x^2$$

where $\text{tr}(D * S) = d_1 + d_2 + d_3 * \det(U) * \det(V)$. If $\det(U) * \det(V) = -1$, the third singular value gets subtracted. This is why the sign correction in S is critical.

Step 9: Compute optimal translation.

$$t^* = \mu_y - s^* * R^* * \mu_x$$

Step 10: Apply the transform to all points in chunk $j+1$.

$$p_{world} = s^* * R^* * p_{local} + t^*$$

5.3 Error Analysis — Drift Accumulation

Sequential stitching introduces accumulated error. Let e_j be the stitching error at chunk j (the angular and translational error in $T_{align}^{(j)}$). The total error after K chunks is bounded by:

$$e_{total} \leq \sum_{j=1}^K e_j$$

In practice the errors are not purely additive (they interact geometrically), but this gives an upper bound. With 88 chunks and a typical per-chunk error of a few degrees in rotation and a few percent in translation, the total drift over a full lap is significant enough that the loop does not close perfectly. This motivates loop closure via global bundle adjustment.

5.4 Loop Closure — Pose Graph Formulation

The complete loop closure problem is formulated as a pose graph optimisation. Define a graph $G = (V, E)$ where:

- **Vertices V** — the set of all camera poses $\{T_i\}$ across all chunks
- **Edges E** — between frames that share a chunk boundary (overlap edges) plus one closure edge between the last and first frame

The optimisation minimises:

$$E_{graph} = \sum_{\{i,j\} \in E} w_{\{ij\}} * || \log(T_i^{-1} * T_j * T_{\hat{ij}}^{-1}) ||^2 + \lambda * || \log(T_K^{-1} * T_0) ||^2$$

where:

- **$T_{\hat{ij}}$** — the relative pose between frames i and j from the reconstruction (the measurement)
- **$w_{\{ij\}}$** — edge weight, proportional to reconstruction confidence
- **$\log(T)$** — the Lie algebra logarithm of a SE(3) element (maps rotations and translations to a 6-vector)
- **λ** — weight of the loop closure constraint

The last term enforces $T_K = T_0$ (the end of the lap coincides with the start). This remains unimplemented and is the primary future direction.

6. Summary of All Mathematical Components

The table below summarises every formula in the pipeline, the variables it operates on, and its role.

- **Bilinear interpolation** — Spatial downscaling of raw frames from 1920x1080 to 1024x576. Variables: pixel positions, scale factors, four-neighbour weights.
 - **Temporal subsampling** — Frame rate reduction. Variables: frame index, stride $k=2$. Constraint: $k \leq \text{fps} / 12$.
 - **Crop operation** — Removal of car body. Variables: frame height H , crop boundary $H/2$. Result: 1024x288 frames.
 - **Chunk construction** — Window decomposition. Variables: $c=20$, $o=5$, $s_t=15$. Result: overlapping windows with 5-frame shared boundaries.
 - **Pairwise pointmap prediction** — DUS3R/MonST3R forward pass. Variables: two input frames, network weights θ . Output: X_a, X_b, C_a, C_b (and M_a, M_b for MonST3R).
 - **Exponential activation** — Depth positivity enforcement. $X = \exp(F_{\text{decoder}})$, $C = \exp(G_{\text{decoder}}) + 1$.
 - **Global alignment energy E** — Confidence-weighted reprojection energy. Variables: T_i in $SE(3)$, σ_{ij} in R^+ , χ . Optimised by Adam with cosine LR decay over 300 iterations.
 - **MST initialisation** — Initial pose estimation from highest-confidence pairs. Graph algorithm $O(|E| \log |V|)$.
 - **Binary confidence mask M** — Threshold C at percentile τ . Output: $M(u,v)$ in $\{0,1\}$. $P_{\text{keep}} = \{\chi(u,v) : M(u,v)=1\}$.
 - **Rigid weld (stitching)** — $T_{\text{align}} = T_{\text{global}} * T_{\text{local}}^{-1}$. Chains chunk reconstructions sequentially into world frame.
 - **Umeyama similarity** — Closed-form (s,R,t) from corresponding camera centres. Steps: mean subtraction, cross-covariance, SVD, sign correction, scale, translation. Complexity $O(n) + O(1)$ for 3x3 SVD.
 - **Pose graph optimisation** — Loop closure formulation (not yet implemented). Minimises sum of squared Lie algebra pose errors plus loop constraint.
-